The Architecture of the Service Discovery Pattern

A High-Level Overview

At a high level, the architecture of the Service Discovery pattern comprises three major components: the Service Registry, the Client Service, and the Server Service.

The Service Registry is a **key-value store** that maintains a record of all active services and their locations. The Client Service is the service that needs to find and interact with other services. The Server Service is the service that the client needs to discover and interact with.

The Client Service

The Client Service is the consumer in this system. Its role is to find and communicate with other services to fulfill its operations. When the client service needs to make a request to another service, it follows these steps:

1. **Service Query**: The client queries the Service Registry to find the location of the Server Service it wants to interact with. This query is typically based on the service name.
2. **Service Selection**: If multiple instances of the Server Service are available, the client selects one. The selection strategy can vary based on the use case - it could be random selection, round-robin, or based on the server load, among others. This step is specific to the client-side discovery model.
3. **Request**: The client then sends a request to the Server Service using the location details it obtained.

The Server Service

The Server Service, on the other hand, is the provider. Its role is to register itself with the Service Registry so other services can find it. Here's what the Server Service typically does:

1. **Service Registration**: When the server starts up, it registers itself with the Service Registry. It provides its name and location details.
2. **Heartbeat**: To ensure the Service Registry has the current status of the service, the server periodically sends a heartbeat signal to the registry. If the Service Registry doesn't receive this signal within a certain timeframe, it assumes that the service has failed and removes it from the registry.

The Service Registry

The Service Registry, as we've discussed, plays a pivotal role in this system. It is responsible for keeping a track of all active services. The Service Registry:

1. **Registers and Deregisters Services**: As we've discussed, it adds services to its store when they come online and removes them when they go offline.
2. **Handles Service Queries**: When a client service sends a service query, the Service Registry looks up its store and returns the location details of the requested service.
3. **Monitors Services**: In some implementations, the Service Registry also monitors the services to ensure their availability. It does this by pinging the services periodically or listening to their heartbeat signals.

Client-side vs. Server-side Discovery in Architecture

As we mentioned before, there are two main flavors of the Service Discovery pattern, which slightly alters this architecture: client-side discovery and server-side discovery.

In the **Client-side Discovery**, the Client Service directly queries the Service Registry and then communicates with the Server Service. In this approach, the Client Service needs to implement the logic for querying the Service Registry and selecting a suitable service from the returned list.

On the other hand, in the **Server-side Discovery**, there's an intermediary, often a load balancer or a router, between the Client Service and the Server Service. The Client Service sends its requests to this intermediary, which queries the Service Registry, selects a suitable Server Service, and forwards the client's request to it. This abstracts away the service discovery details from the Client Service but adds an extra hop in the request path.